

정찬훈

JEONG CHAN-HUN

MMORPG / 온라인 게임 서버 개발

고성능 네트워크 엔진 구현

라이브 서비스 운영·콘텐츠 개발

C++ IOCP 기반 **비동기 네트워크 엔진**을 직접 설계·구현한 게임 서버 프로그래머입니다. 조이시티에서 **프리스타일2** 라이브 서비스와 신작 **프리스타일 리부트**의 시스템·콘텐츠를 개발하며 운영 안정성과 실무 감각을 쌓았습니다. 개인 프로젝트 **DH1_MMORPG**에서는 네트워크 엔진·분산 서버 구조·서버 권위 기반 이동 검증·AOI·UE5 클라이언트까지 **MMORPG 핵심 파이프라인을 1인 개발로 구현**했습니다. CS 지식을 바탕으로 동작 원리를 **코드 수준에서 이해하고 검증**하는 개발자를 지향합니다.

1Y 10M

현업 경력 · 조이시티

2종

라이브·신작 프로젝트

1인

서버·클라이언트 전 영역 구현

현업 경력과 개인 프로젝트 **DH1_MMORPG**(C++ IOCP 게임 서버)를 중심으로 구성했습니다.

01	경력 · 학력 · 자격 EXPERIENCE & BACKGROUND	p.01
02	핵심 역량 요약 PROFILE & POSITIONING	p.02
03	기술 스택 TECHNICAL STACK	p.02
04	프로젝트 — DH1_MMORPG FEATURED PROJECT	p.03
05	고성능 네트워크 엔진 HIGH-PERFORMANCE NETWORKING	p.04
06	메모리 관리 · 멀티 서버 MEMORY & MULTI-SERVER	p.05
07	서버 권위 기반 로직 · 코드 자동화 AUTHORITY & CODEGEN	p.06

현업 경력

● 서버 프로그래머 (사원) 조이시티 · 프리스타일 4팀

2023.05 - 2025.02 (1년 10개월)

- 프리스타일2(라이브 서비스)·프리스타일 리부트(신작 개발) 서버 개발 (C++17 / C++11, MS SQL Server, AWS)
- 시스템 개발 — 레드닷 시스템, 일일·주간 미션 시스템, 장비 추가 능력치 부여, 2차 비밀번호 등 설계·구현
- 콘텐츠 개발 — 주사위·가위바위보·계단오르기 미니게임 이벤트, 주간 미션 상점, 12주년 SLG 미니게임 이벤트
- 라이브 운영 — 실시간 버그 이슈 대응, 레거시 코드 리팩토링, DB 테이블 명세서 작성, 가차 확률 검증용 운영 도구
- 라이브 서비스에서 버그 하나가 다수 유저에 미치는 영향을 체감하며 코드 안정성·가독성을 최우선으로 두는 습관 확립

학력

대학교 한국공학대학교 · 지식서비스공학
2015.03 - 2019.02 졸업

고등학교 유한공업고등학교
2012.03 - 2015.02 졸업

자격 · 교육

자격증 정보처리기사 · 한국산업인력공단
2019.08 취득

교육 이수 프로카데미 16기 서버 프로그래머 과정 졸업
어셈블리 분석·C/C++·네트워크·OS·멀티스레드
·IOCP·MMORPG 서버 구현

주어진 일을 잘 해내는 것을 넘어, 더 나은 구조와 더 깊은 이해를 향해 꾸준히 성장하는 엔지니어가 되겠습니다.

IOCP 비동기 네트워크 엔진·Lock-free 자료구조·메모리 할당기·타이머 스케줄러를 구현하며 OS·네트워크·동시성의 핵심 원리를 코드로 검증했고, 그 위에 서버 사이드 NavMesh 경로탐색·AOI·GameTick·인게임 채팅 등 MMORPG 서버에 필요한 핵심 시스템을 구현했습니다. 이러한 기술적 기반은 조이시티에서 쌓은 라이브 서비스 운영 경험과 안정성 중심의 개발 경험으로 뒷받침됩니다.

게임 서버 개발자로서의 강점 (Why this profile fits)

엔진 레벨 구현 역량

IOCP Overlapped I/O·Lock-free 큐·3계층 메모리 풀·계층형 TimingWheel을 외부 프레임워크 없이 구현. 서버가 멈추거나 느려지는 원인을 엔진 레벨에서 진단·튜닝합니다.

게임 적용 · 동접 부하·응답 지연·메모리 누수 대응

게이트웨이 중심 구조

클라이언트는 GatewayServer 한 곳에만 접속 (단일 커넥션)하고, Gateway가 내부 World·Realm으로 패킷을 릴레이하는 구조를 설계·구현했습니다.

게임 적용 · 단일 접속점·내부 토폴로지 은닉

라이브 서비스 실무

프리스타일2·리부트에서 미션·레드닷 시스템과 미니게임 이벤트를 개발하고, 라이브 버그 대응·레거시 리팩토링을 수행하며 실제 유저에게 영향을 주는 코드의 안정성을 최우선으로 다뤘습니다.

게임 적용 · 게임 콘텐츠·시스템 개발, 라이브 이슈 경험

03 Technical Stack

Languages

주력 / 보조

C++20 / C++17 / C++11 C# SQL Assembly (분석)

Game Server

게임 서버 시스템

서버 권위 기반 로직 인게임 채팅 Gateway 기반 패킷 릴레이 서버 사이드 NavMesh AOI / GridManager

Networking

저수준 I/O

Windows IOCP SendBufferAllocator HTTPS / REST Custom Network Engine

Concurrency

동시성·자료구조

Actor Model ScopedActor Lock-free Queue/Stack atomic / CAS 3-Tier Memory Pool TimingWheel

Multi-Server

분산·상태 동기화

Redis (Session / Pub/Sub) Gateway 로드밸런싱 크로스 서버 채팅

Backend

서비스 계층

ASP.NET Core EF Core (Migrations) REST API Background Service

Client

클라이언트 통합

Unreal Engine 5 (C++) Custom Network Engine 이식

Build / Tooling

빌드·자동화

PacketGenerator (.NET) protoc MSBuild / UBT vcpkg BuildScripts (.bat)

* 표기된 기술은 실제 코드로 구현했거나 실무에서 사용한 항목 위주로 정리했습니다.

C++20 기반 IOCP 커스텀 네트워크 엔진(CppNetEngine)을 설계·구현하고, 이를 GatewayServer·WorldServer·RealmServer와 UE5 클라이언트가 공유하도록 구성하고, 인증 기능은 C# 기반 LoginServer로 분리한 MMORPG 프로젝트입니다. I/O 모델·동시성·메모리 풀·타이머 스케줄러부터 분산 서버 아키텍처, 프로토콜 자동화, UE5 클라이언트 통합까지 전 영역을 직접 구현했습니다.

유형 / 기간 개인 프로젝트 (1인 개발) · 2026.01 ~ 진행 중

구성 C++20 네트워크 엔진 · C# ASP.NET Core 인증 서버 · Protobuf 프로토콜 · UE5 클라이언트까지 1인 구현

저장소 www.github.com/jch5158/DH1_MMORPG

시연 영상 www.youtube.com/watch?v=RJqDSDwSnE

2

독립 IOCP 스케줄러
(Network / Actor)

3계층

메모리 풀
TLS → 글로벌 → OS

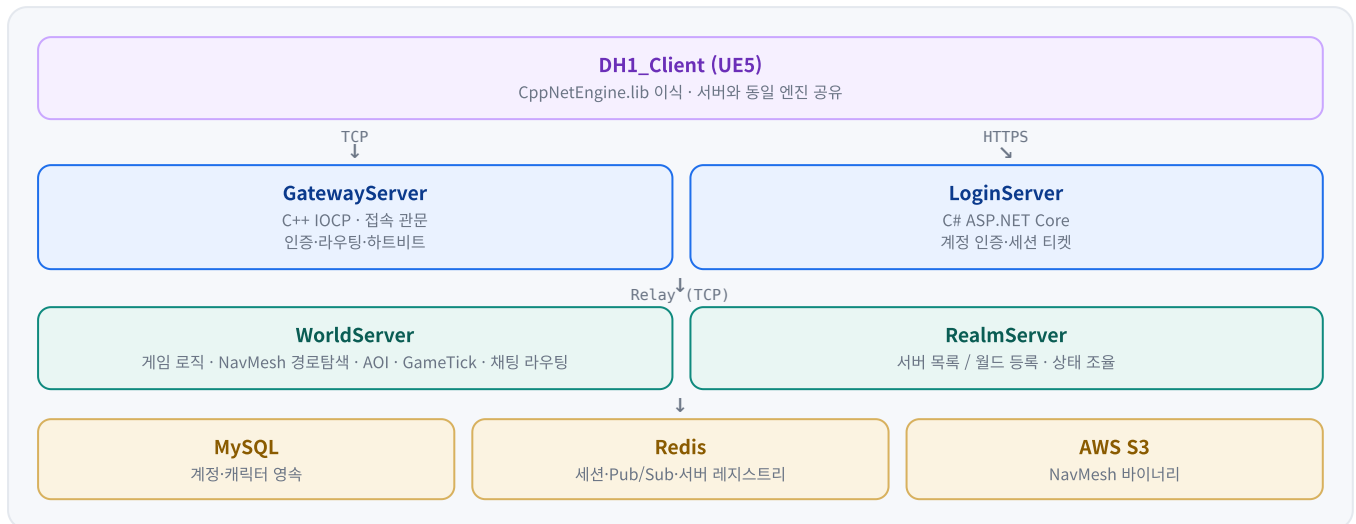
3레벨

계층형 TimingWheel
256·256·128 슬롯

4

분산 서버 역할 분리
+ UE5 클라이언트

시스템 아키텍처 (Distributed Topology)



각 C++ 서버는 CppNetEngine을 static lib로 링크하며, 클라이언트(UE5)도 동일 엔진을 이식하여 서버·클라이언트가 단일 네트워크 스택을 공유합니다.

주요 기여 & 성과

커스텀 네트워크 엔진

IOCP 비동기 I/O · Actor 모델 · Lock-free 자료구조 · 3계층 메모리 풀 · TimingWheel을 포함한 C++ 서버 엔진을 외부 프레임워크 없이 구현.

분산 서버 아키텍처

역할별 4개 서버 분리 + Relay 패턴 + Redis 세션으로 멀티 서버 구조 구현. LoginServer는 Gateway별 세션 수(Redis 조회)를 비교해 **최소 연결 노드**로 분배.

UE5 클라이언트 구현

CppNetEngine을 UE5에 이식해 서버·클라이언트가 동일 네트워크 스택을 공유. 로그인·캐릭터 생성 UI, 접속·인증 흐름, 패킷 송수신, 캐릭터 이동·서버 동기화, 인게임 채팅까지 직접 구현.

프로토콜 자동화 도구

.proto 커스텀 옵션 기반 **PacketGenerator(.NET)**를 직접 개발해 역할별 패킷 코드·디스패처를 자동 생성, 서버·클라이언트 프로토콜을 동기화.

엔진의 핵심은 **소켓 I/O와 게임 로직을 역할별로 컴포넌트화한 이중 IOCP 스케줄러**입니다. 각 스케줄러는 독립된 IOCP 핸들과 스레드 풀을 가져 서로 간섭하지 않으며, 메시지는 Lock-free 큐로만 전달됩니다.

NetworkScheduler 소켓 I/O 전담

Overlapped I/O 완료를 `GetQueuedCompletionStatus`로 수신 → 패킷 역직렬화·송신 완료 처리. 디스패치 루프마다 `TimingWheel`을 Tick하여 하트비트·세션 타임아웃을 함께 스케줄링.

ActorScheduler 게임 로직 · DB 비동기

소켓 없이 `PostQueuedCompletionStatus`로 Actor를 깨우는 **수동 완료 큐**. 게임 로직뿐 아니라 **DB 쿼리도 전용 Actor에 Post**해 비동기 처리하며, 라운드 로빈 / 라우팅 키(유저별 순서 보장)로 분배합니다.

Actor 모델 — 락 없는 단일 소유 동시성

각 Actor는 **단일 atomic<bool> 플래그**의 `TryAcquire / Release`로 보호되어, 한 번에 하나의 디스패치 스레드만 진입합니다. 멀티 객체를 동시에 다뤄야 하는 경우 **ScopedActor**가 ID 순으로 정렬해 일괄 잠금하여 **데드락을 원천 차단**합니다.

```
// ScopedActor - 고정 순서 잠금 + 단계적 백오프로 데드락/라이브락 방지
bool ScopedActor::TryAcquire() {
    for (int32 retry = 0; retry < retryLimit; ++retry) {
        if (tryAcquireAll()) return true; // ID 순 정렬 후 일괄 Acquire
        Release();                       // 실패 시 역순 전체 해제
        if (retry < 10) _mm_pause();      // 스핀
        else if (retry < 50) std::this_thread::yield(); // 양보
        else std::this_thread::sleep_for(1ms); // 백오프
    }
}
```

Lock-free 자료구조 — ABA 방지 설계

LockFreeQueue

- Dummy-head 연결 리스트 + CAS 기반 원자적 갱신
- **Node16**(포인터 + 64-bit 버전 카운터)로 **ABA 문제 차단**
- ActorMailbox · SendBuffer 큐의 기반

LockFreeStack

- Treiber 스택 + Node16 wide-CAS
- top 버전 카운터로 ABA 방지
- 세션 풀 · ConnectionPool 인덱스 관리

설계 원칙 · `alignas(hardware_destructive_interference_size)`로 head/tail/count를 별도 캐시 라인에 배치해 **false sharing**을 제거하고, 노드 할당은 TLS 캐시(`TlsObjectPool`)에서 처리하여 atomic 연산 오버헤드를 최소화했습니다.

계층형 TimingWheel — O(1) 타이머 스케줄링

하트비트·세션 타임아웃·지연 콜백을 위한 타이머를 **3레벨 계층 구조(256 / 256 / 128 슬롯)**로 구현했습니다. 표준 우선순위 큐 대비 **삽입·만료가 상수 시간**에 처리되며, 디스패치 루프에 통합되어 별도 타이머 스레드가 없습니다.

3계층 메모리 할당 전략

빈번한 패킷 송수신에서 할당 비용과 경합을 줄이기 위해, **스레드 로컬 캐시 → 글로벌 Lock-free 풀 → mimalloc(OS)**로 이어지는 3계층 전략을 설계했습니다.



안전성 · 무결성

각 노드에 체크섬 쿼키(0xDEAD...BEEF)를 심어 **double-free·메모리 손상**을 런타임에 감지. **MemoryPool**은 블록마다 소속 Chunk 역참조 포인터를 내장해 **Free()**를 O(1)로 처리.

SendBufferAllocator

요청 크기가 **4096B** 이하면 256B, 초과 시 **4096B** 단위로 올림 정렬(최대 65536B)해 **SharedPtr<NetSendBuffer>** 발급. 패킷 크기 편차로 인한 할당 단편화를 억제.

게임 서버 관점 · 초당 수많은 패킷이 오가는 MMORPG에서 **할당 비용·단편화·메모리 손상**은 응답 지연과 크래시로 직결됩니다. 이를 엔진 레벨에서 통제하는 구조입니다.

멀티 서버 · 수평 확장 설계

최소 연결 로드밸런싱

각 Gateway가 자신의 세션 수·상태를 Redis에 등록(**Gateway:***). LoginServer의 **GatewaySelector**가 온라인 게이트웨이 중 **현재 연결이 가장 적은 노드**를 선택해 접속을 분배.

게임 적용 · 접속 폭주 시 서버 간 부하 분산·증설

Relay 패턴 · 서버 간 패킷 중계

Gateway ↔ World 간 **RELAY_TO_CLIENT / RELAY_TO_WORLD**로 패킷을 중계. 클라이언트는 단일 접속점만 알면 되고, 내부 서버는 역할에만 집중.

게임 적용 · 접속 관문 분리·내부 토폴로지 은닉

범위 기반 채팅 라우팅

채팅을 **LOCAL(AOI 범위)·WORLD(월드 전체)·REALM**으로 구분해 패킷 릴레이. REALM 채팅은 **RealmServer**를 통해 여러 World로 S2S 전파.

게임 적용 · 근거리·월드·렐름 범위별 채팅 전파

세션 신뢰성 · 자동 정리

양방향 하트비트 + Redis 세션 TTL + 중복 로그인 Kick(Redis Pub/Sub) + **SessionReaper** 자동 정리로 **좀비 세션·자원 누수**를 차단.

게임 적용 · 끊긴 접속 정리·중복 로그인·계정 보안

서버 사이드 NavMesh — 서버 권위 기반 경로 탐색

이동 검증을 클라이언트가 아닌 서버에서 수행하기 위해, **UE5 Detour 라이브러리**를 서버에 이식하여 서버가 경로 계산과 이동 검증의 최종 권한을 가집니다. NavMesh 바이너리는 **AWS S3에서 로드**하며, DB(world_navmesh_source)로 버전·맵별 경로를 관리하고, 로딩 실패 시 **fail-fast로 기동을 중단**해 잘못된 상태로의 서비스 진입을 막습니다.

FindPath

dtNavMeshQuery 기반 start→end 경로 탐색, GameTick에서 경로 추종.

AOI · GridManager

맵을 격자 셀로 분할(셀 크기 설정값). 셀 경계 이동 시 **Enter/Leave 자동 브로드캐스트**로 관심 영역만 동기화.

S3 + IAM

최소 권한(**s3:GetObject**) IAM Role로 NavMesh 다운로드. 운영 보안을 고려한 권한 설계.

AOI(Area of Interest)는 “플레이어의 시야 안에 들어온 대상만 추려 전송”하는 구조로, 대규모 동접 환경에서 위치·상태 동기화 트래픽을 크게 줄이는 MMORPG의 핵심 기법입니다.

PacketGenerator — 프로토콜 코드 자동 생성

.proto에 커스텀 옵션(packet_id · sender · receiver)으로 메타데이터를 선언하면, **역할(Role)별 수신 등록 코드·송신 헬퍼·통합 디스패처·PacketId enum이 자동 생성**됩니다. 현재 프로젝트 역할에 해당하는 메시지만 골라 코드를 만들고, 클라이언트가 실제로 쓰는 proto만 UE 모듈로 복사하여 **서버·클라이언트 프로토콜을 항상 동기화**합니다.

```
// .desc(descriptor) → 역할별 C++ 코드 자동 생성 파이프라인
.proto → protoc → .pb.h / .desc → PacketGenerator.exe (.NET)
      ↓
PacketId.h · {Role}PacketHandler.h · UE Protocol/*.pb.cpp (클라 복사)
```